

POLITECHNIKA WARSZAWSKA

WYDZIAŁ ELEKTRONIKI I TECHNIK INFORMACYJNYCH

PROGRAMOWANIE RÓWNOLEGŁE I ROZPROSZONE

Sprawozdanie

Autorzy:

Mateusz CHMURZYŃSKI

Filip KIERNOZEK

Bartłomiej KRAJEWSKI

Bartosz SĘDZIKOWSKI

13 stycznia 2026

Spis treści

1	Wprowadzenie	2
1.1	Zmiany w stosunku do etapu 1	2
1.2	Etapy realizacji projektu	2
2	Metoda optymalizacji: Particle Swarm Optimization (PSO)	3
2.1	Zasada działania algorytmu	3
2.2	Implementacja sekwencyjna i implementacje równoległe	3
3	Zadania optymalizacji	5
3.1	Zadanie 1: Funkcja Schwefel	5
3.2	Zadanie 2: Funkcja Rosenbrock	5
4	Kryteria zatrzymania	5
5	Parametry eksperymentu	5
6	Wyniki eksperymentów	6
6.1	Wersja sekwencyjna + wersja równoległa procesy - etap 1	6
6.1.1	Zadanie 1: Funkcja Schwefel	6
6.1.2	Zadanie 2: Funkcja Rosenbrock	6
6.2	Wersja sekwencyjna	6
6.2.1	Zadanie 1: Funkcja Schwefel	7
6.2.2	Zadanie 2: Funkcja Rosenbrock	7
6.3	Wersja równoległa - procesy	7
6.3.1	Zadanie 1: Funkcja Schwefel	7
6.3.2	Zadanie 2: Funkcja Rosenbrock	8
6.4	Wersja równoległa - wątki	8
6.4.1	Zadanie 1: Funkcja Schwefel	8
6.4.2	Zadanie 2: Funkcja Rosenbrock	9
6.5	Wersja równoległa - CUDA	9
6.5.1	Zadanie 1: Funkcja Schwefel	9
6.5.2	Zadanie 2: Funkcja Rosenbrock	10
6.6	Wersja równoległa - MPI	10
6.6.1	Zadanie 1: Funkcja Schwefel	10
6.6.2	Zadanie 2: Funkcja Rosenbrock	10
7	Wnioski i analiza	11
7.1	Wnioski i analiza z etapu 1	11
7.1.1	Analiza wydajności i przyspieszenia - etap 1	11
7.1.2	Zbieżność algorytmu w wysokich wymiarach - etap 1	11
7.2	Wnioski i analiza - etap 2	11
8	Wizualizacja zbieżności algorytmu (n=2)	13
8.1	Funkcja Schwefela	13
8.2	Funkcja Rosenbrocka	14

1 Wprowadzenie

Niniejsze sprawozdanie dotyczy pierwszego etapu projektu z przedmiotu Programowanie równoległe i rozproszone. Celem projektu jest implementacja sekwencyjnej i równoległej wersji algorytmu optymalizacji, a następnie ocena przyspieszenia obliczeń w zależności od stopnia zrównoleglenia. Ponieważ oba etapy stanowią integralną część projektu, niniejsze sprawozdanie obejmuje oba etapy.

1.1 Zmiany w stosunku do etapu 1

Po pierwszym etapie uwzględniliśmy otrzymane uwagi oraz wprowadziliśmy dodatkowe zmiany:

- Funkcje Rosenbrock i Schwefel zostały przepisane tak, aby przyjmowały macierze i wykonywały obliczenia na macierzach. Wyeliminowaliśmy w ten sposób funkcję `apply_along_axis`, która znacząco opóźniała wykonanie programów.
- W celu usprawnienia warunku stopu zastosowano strukturę deque o ograniczonym rozmiarze pełniącą rolę przesuwającego okna (sliding window) dla wartości globalnego optimum. Pozwala to monitorować brak poprawy w ostatnich iteracjach bez potrzeby przechowywania całej historii wyników.
- Zoptymalizowano implementację PSO poprzez zastosowanie operacji in-place w NumPy. W efekcie ograniczono liczbę kopii macierzy tworzonej w trakcie aktualizacji współczynników i położenia cząstek, co redukuje koszty alokacji oraz zwiększa przepustowość obliczeń wektorowych.
- Uzupełniliśmy eksperymenty o większą liczbę rdzeni - w pierwszym etapie było to tylko 6 rdzeni, teraz jest to 2 i 4 rdzenie (4 rdzenie to maksimum ponieważ eksperymenty były przeprowadzane na innej maszynie).

1.2 Etapy realizacji projektu

W ramach drugiego etapu projektu zrealizowano następujące zadania:

1. Uwzględnienie uwag i wprowadzenie zmian w wersji sekwencyjnej oraz równoległej z wykorzystaniem procesów.
2. Implementacja algorytmu PSO w wersji równoległej na maszynie z pamięcią wspólną z wykorzystaniem modułu CuPy w Pythonie.
3. Implementacja algorytmu PSO w wersji równoległej z wykorzystaniem wątków w Pythonie.
4. Implementacja algorytmu PSO w wersji równoległej z wykorzystaniem MPI w Pythonie.
5. Implementacja algorytmu PSO w wersji równoległej z wykorzystaniem Spark w Pythonie - implementacja nie weszła do rozwiązania ze względu na duże czasy wykonania eksperymentów.
6. Przeprowadzenie eksperymentów dla różnej wielkości zadania dla wszystkich algorytmów (również implementowanych w etapie 1) oraz ocena przyspieszenia obliczeń w zależności od rozmiaru problemu.

2 Metoda optymalizacji: Particle Swarm Optimization (PSO)

Algorytm **Particle Swarm Optimization (PSO)** jest ewolucyjną techniką optymalizacji, inspirowaną zachowaniem stad ptaków lub ławic ryb. Jest to metoda heurystyczna, wykorzystywana do znajdowania globalnych minimów funkcji celu w przestrzeni wielowymiarowej.

2.1 Zasada działania algorytmu

W algorytmie PSO, populacja potencjalnych rozwiązań, zwana **rojem (swarm)**, składa się z pojedynczych cząstek (**particles**). Każda cząstka w roju porusza się w przestrzeni poszukiwań z określoną prędkością, dostosowując swoją trajektorię na podstawie własnego najlepszego doświadczenia oraz najlepszego doświadczenia całego roju.

Każda cząstka i charakteryzuje się:

- **Pozycją** $x_i(t)$ w przestrzeni poszukiwań, gdzie t to numer iteracji.
- **Prędkością** $v_i(t)$.
- **Najlepszą dotychczasową pozycją osobistą** $pbest_i$, która dała najniższą wartość funkcji celu.

Dodatkowo, algorytm śledzi **najlepszą globalną pozycję** $gbest$, która jest najlepszą pozycją spośród wszystkich $pbest_i$ w roju.

W każdej iteracji, prędkość i pozycja cząstki są aktualizowane według następujących równań:

$$v_i(t+1) = wv_i(t) + c_1r_1(pbest_i - x_i(t)) + c_2r_2(gbest - x_i(t)) \quad (1)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2)$$

gdzie:

- w – **współczynnik bezwładności**, kontroluje wpływ poprzedniej prędkości.
- c_1 – **współczynnik kognitywny**, dążenie do własnego $pbest$.
- c_2 – **współczynnik społeczny**, dążenie do globalnego $gbest$.
- r_1, r_2 – losowe liczby z zakresu $[0, 1]$.

2.2 Implementacja sekwencyjna i implementacje równoległe

Główna różnica między implementacją sekwencyjną a równoległą dotyczy sposobu obliczania wartości funkcji celu dla całej populacji.

- **Wersja sekwencyjna:** W pliku `sequential/main.py` funkcje benchmarkowe przyjmują macierze wszystkich cząstek w jednym wątku i obliczają ich wartości funkcji. Zaimplementowano klasyczną wersję algorytmu, w której wszystkie operacje sterujące i obliczeniowe realizowane są liniowo w obrębie jednego procesu głównego. Cała populacja cząstek jest inicjalizowana jednorazowo i przechowywana w pamięci jako zwarte macierze biblioteki NumPy. W każdej iteracji aktualizacja prędkości oraz pozycji wszystkich cząstek odbywa się sekwencyjnie bez narzutu związanego z komunikacją międzyprocesową.

- **Wersja równoległa - procesy:** W pliku `parallel_processes/main.py` zastosowano podejście oparte na `multiprocessing`, które pozwala podzielić obciążenie obliczeniowe między wiele rdzeni.
 1. Populacja cząstek jest dzielona na mniejsze części (*chunks*) przez funkcję `chunkify`.
 2. Każda część trafia do osobnego procesu w puli (`Pool`).
 3. Funkcja `eval_block` jest wykonywana równolegle na wielu rdzeniach.
 4. Otrzymane wyniki są następnie zbierane i łączone w procesie głównym.
- **Wersja równoległa - wątki:** W pliku `parallel_threads/main.py` zastosowano podejście oparte na module `threading` oraz współdzielonej pamięci, zoptymalizowane pod kątem Pythona 3.13 w trybie *no-GIL*.
 1. Wykorzystano mechanizm *shared memory*, gdzie wątki operują bezpośrednio na fragmentach głównych macierzy NumPy, eliminując narzut IPC (Inter-Process Communication).
 2. Synchronizacja iteracji realizowana jest za pomocą obiektu `threading.Barrier`, zapewniającego spójność kroku algorytmu.
 3. Funkcja `pso_worker` przetwarza przypisany zakres cząstek w sposób ciągły, minimalizując alokację nowych obiektów.
 4. Środowisko uruchomieniowe zostało skompilowane z flagą `--disable-gil`, co umożliwia rzeczywiste, równoległe wykonywanie kodu bajtowego Pythona na wielu rdzeniach.
- **Wersja równoległa - CUDA:** W pliku `parallel_cuda/main.py` zastosowano podejście oparte na wykorzystaniu mocy obliczeniowej karty graficznej poprzez środowisko programistyczne CUDA, a dokładniej bibliotkę CuPy w języku Python.
 1. Wszystkie obliczenia (operacje macierzowe, funkcje `argmin()` itp.) wykonywane są na karcie graficznej dzięki bibliotece CuPy.
 2. Do obliczenia kolejnych prędkości wykorzystano własny rdzeń CUDA, który wykonuje operacje element po elemencie (`cupy.ElementwiseKernel`), tak jak w głównym algorytmie.
 3. Końcowe wyniki obliczeń są przenoszone na CPU poprzez funkcję `cupy.asnumpy()`.
 4. Brak API do niskopoziomowego zarządzania pamięcią na karcie graficznej oraz synchronizacji wątków.
- **Wersja równoległa - MPI:** W pliku `parallel_mpi/main.py` wykorzystano standard Message Passing Interface (MPI) oraz bibliotekę `mpi4py`. W tym podejściu pamięć nie jest współdzielona – każdy proces posiada własną, niezależną pamięć operacyjną.
 1. Całkowity rój cząstek jest dzielony na podzbiory przydzielone do poszczególnych procesów na podstawie ich unikalnego numeru (`comm.rank()`).
 2. Każdy proces niezależnie aktualizuje prędkości i pozycje oraz oblicza wartości funkcji celu tylko dla przypisanych mu cząstek.
 3. W każdej iteracji procesy wymieniają się informacjami o najlepszych lokalnych wynikach za pomocą funkcji `comm.allgather()`, aby wspólnie wyłonić lidera całego roju (`gbest`) przy minimalnym narzucie komunikacyjnym..

3 Zadania optymalizacji

W projekcie wykorzystano dwie funkcje testowe zaimplementowane w pliku `benchmark_functions.py`.

3.1 Zadanie 1: Funkcja Schwefel

Funkcja Schwefela jest wielomodalna, z wieloma lokalnymi minimami, co sprawia, że jest trudna dla algorytmów optymalizacji.

$$f(\mathbf{x}) = 418.9829n - \sum_{i=1}^n \left[x_i \sin(\sqrt{|x_i|}) \right]$$

Ograniczenia: $-100 \leq x_i \leq 500$. Optimum globalne $f_{min} = 0$ znajduje się w punkcie $x_i \approx 420.9687$.

3.2 Zadanie 2: Funkcja Rosenbrock

Funkcja Rosenbrocka, znana jako "dolina", posiada minimum w płaskim, wąskim parabolicznym obszarze.

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]$$

Ograniczenia: $-10 \leq x_i \leq 10$. Optimum globalne $f_{min} = 0$ znajduje się w punkcie $x_i = 1$.

4 Kryteria zatrzymania

Zaimplementowano dwa warunki stopu w pliku `stop_conditions.py`:

1. **Znane optimum:** Algorytm kończy działanie, gdy norma różnicy między najlepszą znalezioną pozycją a znanym optimum jest mniejsza od ε_{opt} ($\|g_{best} - x_{opt}\| \leq \varepsilon$).
2. **Brak poprawy:** Algorytm kończy działanie, gdy przez m iteracji zmiana wartości funkcji celu jest mniejsza niż $\varepsilon_{no_improve}$.

5 Parametry eksperymentu

Wszystkie testy wydajnościowe przeprowadzono przy użyciu jednolitych parametrów dla obu funkcji celu oraz wszystkich wymiarów przestrzeni. Pozwoliło to na rzetelne porównanie czasów wykonania.

Parametry algorytmu PSO:

- **Rozmiar roju (swarm_size):** 1000 cząstek.
- **Maksymalna liczba iteracji:** 10000.
- **Współczynniki:** $w = 0.7$ (bezładność), $c_1 = 1.5$ (komponent kognitywny), $c_2 = 1.5$ (komponent społeczny).

Kryteria zatrzymania:

1. **Kryterium 1 (Znane optimum):** Stop, gdy $\|g_{best} - x_{opt}\| \leq 10^{-3}$.
2. **Kryterium 2 (Brak poprawy):** Stop, gdy brak poprawy o 10^{-6} przez 50 iteracji.

6 Wyniki eksperymentów

6.1 Wersja sekwencyjna + wersja równoległa procesy - etap 1

Pozostawiamy wyniki z etapu pierwszego w sprawozdaniu. Wyniki aktualnych wersji sekwencyjnych i równoległej z procesami będą pokazane w następnych sekcjach.

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z 6-rdzeniowym procesorem. Tabela zawiera czasy wykonania dla wersji sekwencyjnej (T_{seq}) i równoległej (T_{par}) oraz obliczone przyspieszenie $S = T_{seq}/T_{par}$.

6.1.1 Zadanie 1: Funkcja Schwefel

Tabela 1: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	Iteracje	Najlepsze $f(x)$	T_{seq} [s]	T_{par} [s]	Przyspieszenie (S)
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.96 s	0.75 s	1.28
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	1.22 s	0.86 s	1.42
50	Znane Opt.*	10000	6726.75	90.95 s	47.96 s	1.90
50	Brak Popr.	357	6726.75	3.61 s	2.87 s	1.26
100	Znane Opt.*	10000	11854.13	114.80 s	67.20 s	1.71
100	Brak Popr.	1024	11854.13	13.06 s	10.48 s	1.25

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.1.2 Zadanie 2: Funkcja Rosenbrock

Tabela 2: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	Iteracje	Najlepsze $f(x)$	T_{seq} [s]	T_{par} [s]	Przyspieszenie (S)
10	Znane Opt.*	10000	5.42	99.16 s	39.12 s	2.53
10	Brak Popr.	191	5.42	1.99 s	1.03 s	1.93
50	Znane Opt.*	10000	115.20	110.38 s	54.96 s	2.01
50	Brak Popr.	9003	115.21	101.85 s	54.14 s	1.88
100	Znane Opt.*	10000	119.96	138.10 s	97.86 s	1.41
100	Brak Popr.*	10000	119.96	150.53 s	106.02 s	1.42

*Algorytm osiągnął limit iteracji.

6.2 Wersja sekwencyjna

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z 4-rdzeniowym procesorem. Tabela zawiera czasy wykonania dla wersji sekwencyjnej (T_{seq}).

6.2.1 Zadanie 1: Funkcja Schwefel

Tabela 3: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,sec}$	Najlepsze $f(x)$	T_{seq} [s]	$T_{seq}/i_{max,sec}$ [s]
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.09	$7.21 \cdot 10^{-4}$
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	0.10	$5.90 \cdot 10^{-4}$
50	Znane Opt.*	10000	6726.75	30.73	$3.07 \cdot 10^{-3}$
50	Brak Popr.	357	6726.75	1.03	$2.90 \cdot 10^{-3}$
100	Znane Opt.*	10000	11854.13	55.57	$5.56 \cdot 10^{-3}$
100	Brak Popr.	1024	11854.13	5.97	$5.83 \cdot 10^{-3}$

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.2.2 Zadanie 2: Funkcja Rosenbrock

Tabela 4: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,sec}$	Najlepsze $f(x)$	T_{seq} [s]	$T_{seq}/i_{max,sec}$ [s]
10	Znane Opt.*	10000	5.42	4.11	$4.11 \cdot 10^{-4}$
10	Brak Popr.	191	5.42	0.08	$4.18 \cdot 10^{-4}$
50	Znane Opt.*	10000	115.20	21.50	$2.15 \cdot 10^{-3}$
50	Brak Popr.	9003	115.21	19.47	$2.16 \cdot 10^{-3}$
100	Znane Opt.*	10000	119.96	45.52	$4.55 \cdot 10^{-3}$
100	Brak Popr.*	10000	119.96	42.22	$4.22 \cdot 10^{-3}$

*Algorytm osiągnął limit iteracji.

6.3 Wersja równoległa - procesy

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z 4-rdzeniowym procesorem. Tabela zawiera czasy wykonania dla wersji równoległej z wykorzystaniem procesów ($T_{proc,r}$) wykorzystująca r rdzeni oraz obliczone przyspieszenie $S(n, r) = T_{seq}/T_{proc,r}$.

6.3.1 Zadanie 1: Funkcja Schwefel

Tabela 5: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,proc}$	Najlepsze $f(x)$	$T_{proc,2}$ [s]	$T_{proc,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.27	0.32	0.32	0.27
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	0.76	0.32	0.13	0.30
50	Znane Opt.*	10000	6726.75	40.43	45.59	0.76	0.67
50	Brak Popr.	357	6726.75	1.31	2.34	0.79	0.44
100	Znane Opt.*	10000	11854.13	75.51	76.44	0.74	0.73
100	Brak Popr.	1024	11854.13	7.72	7.75	0.77	0.77

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.3.2 Zadanie 2: Funkcja Rosenbrock

Tabela 6: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,proc}$	Najlepsze $f(x)$	$T_{proc,2}$ [s]	$T_{proc,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.*	10000	5.42	12.96	20.97	0.32	0.20
10	Brak Popr.	191	5.42	0.27	0.39	0.30	0.20
50	Znane Opt.*	10000	115.20	32.68	39.19	0.66	0.55
50	Brak Popr.	9003	115.21	27.74	37.78	0.70	0.52
100	Znane Opt.*	10000	119.96	70.40	77.37	0.65	0.59
100	Brak Popr.*	10000	119.96	67.22	74.23	0.63	0.57

*Algorytm osiągnął limit iteracji.

6.4 Wersja równoległa - wątki

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z 4-rdzeniowym procesorem. Tabela zawiera czasy $T_{thr,r}$ wykonania dla wersji równoległej z wykorzystaniem r wątków oraz obliczone przyśpieszenie $S(n, r) = \frac{T_{seq}}{i_{max,sec}} / \frac{T_{thr,n}}{i_{max,n}}$, gdzie $i_{max,sec}$ to końcowa liczba iteracji algorytmu sekwencyjnego, natomiast $i_{max,r}$ to końcowa liczba iteracji algorytmu równoległego na r rdzeniach.

6.4.1 Zadanie 1: Funkcja Schwefel

Tabela 7: Wyniki dla funkcji Schwefela (Swarm Size = 1000, wątki=2)

Wymiar (n)	Kryterium	$i_{max,2}$	Najlepsze $f(x)$	$T_{thr,2}$ [s]	$T_{thr,2}/i_{max,2}$ [s]	$S(n, 2)$
10	Znane Opt.	4074	145.83	11.85	$5.81 \cdot 10^{-2}$	0.01
10	Brak Popr.	167	145.83	8.68	$5.20 \cdot 10^{-2}$	0.01
50	Znane Opt.*	10000	6046.14	36.17	$3.62 \cdot 10^{-3}$	0.85
50	Brak Popr.	375	5855.81	11.90	$3.18 \cdot 10^{-2}$	0.09
100	Znane Opt.*	10000	15217.20	49.57	$4.96 \cdot 10^{-2}$	1.12
100	Brak Popr.	852	15715.70	13.81	$1.63 \cdot 10^{-2}$	0.36

*Algorytm osiągnął limit iteracji nie znajdując optimum zadaną dokładnością.

Tabela 8: Wyniki dla funkcji Schwefela (Swarm Size = 1000, wątki=4)

Wymiar (n)	Kryterium	$i_{max,4}$	Najlepsze $f(x)$	$T_{thr,4}$ [s]	$T_{thr,4}/i_{max,4}$ [s]	$S(n, 4)$
10	Znane Opt.	2110	72.91	12.29	$6.21 \cdot 10^{-2}$	0.01
10	Brak Popr.	164	72.92	8.94	$5.41 \cdot 10^{-2}$	0.01
50	Znane Opt.*	10000	5956.52	25.10	$2.51 \cdot 10^{-3}$	1.22
50	Brak Popr.	408	25.02	11.13	$2.73 \cdot 10^{-2}$	0.11
100	Znane Opt.*	10000	15580.81	37.92	$3.79 \cdot 10^{-3}$	1.47
100	Brak Popr.	795	15237.43	6.47	$8.53 \cdot 10^{-3}$	0.68

*Algorytm osiągnął limit iteracji nie znajdując optimum zadaną dokładnością.

6.4.2 Zadanie 2: Funkcja Rosenbrock

Tabela 9: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000, wątki=2)

Wymiar (n)	Kryterium	$i_{max,2}$	Najlepsze $f(x)$	$T_{thr,2}$ [s]	$T_{thr,2}/i_{max}$ [s]	$S(n,2)$
10	Znane Opt.	6953	$5.18 \cdot 10^{-7}$	13.58	$2.56 \cdot 10^{-3}$	0.16
10	Brak Popr.	3020	0.80	13.07	$4.63 \cdot 10^{-3}$	0.09
50	Znane Opt.*	10000	2268.32	27.83	$2.78 \cdot 10^{-3}$	0.77
50	Brak Popr.	9480	29.21	25.99	$2.76 \cdot 10^{-3}$	0.78
100	Znane Opt.*	10000	408.20	44.02	$4.40 \cdot 10^{-3}$	1.03
100	Brak Popr.*	10000	304.12	42.14	$4.21 \cdot 10^{-3}$	1.00

*Algorytm osiągnął limit iteracji.

Tabela 10: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000, wątki=4)

Wymiar (n)	Kryterium	$i_{max,4}$	Najlepsze $f(x)$	$T_{thr,4}$ [s]	$T_{thr,4}/i_{max,4}$ [s]	$S(n,4)$
10	Znane Opt.	7379	$5.64 \cdot 10^{-8}$	16.85	$2.69 \cdot 10^{-3}$	0.15
10	Brak Popr.	3844	$2.24 \cdot 10^{-4}$	11.52	$2.86 \cdot 10^{-3}$	0.14
50	Znane Opt.*	10000	57.83	32.94	$3.29 \cdot 10^{-3}$	0.65
50	Brak Popr.	9470	407.16	33.44	$3.54 \cdot 10^{-3}$	0.61
100	Znane Opt.*	10000	2112.82	42.59	$4.26 \cdot 10^{-3}$	1.07
100	Brak Popr.*	10000	2096.77	34.31	$3.43 \cdot 10^{-3}$	1.23

*Algorytm osiągnął limit iteracji.

6.5 Wersja równoległa - CUDA

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z kartą graficzną NVIDIA GTX 1050 o pamięci dedykowanej 4.0 GB i współdzielonej 3.9 GB. Tabela zawiera czasy wykonania dla wersji równoległej z wykorzystaniem pakietu CuPy (T_{cupy}) oraz obliczone przyspieszenie. Ze względu na inne losowanie liczb w algorytmie przez pakiet CuPy wyniki nie pokrywają się z wersją sekwencyjną. Dlatego do obliczenia przyspieszenia wykorzystaliśmy liczbę iteracji na sekundę $S(n) = \frac{T_{seq}}{i_{max,seq}} / \frac{T_{cuda}}{i_{max,cuda}}$.

6.5.1 Zadanie 1: Funkcja Schwefel

Tabela 11: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,cuda}$	Najlepsze $f(x)$	T_{cuda} [s]	$T_{cuda}/i_{max,cuda}$ [s]	$S(n)$
10	Znane Opt.	157	$1.27 \cdot 10^{-4}$	0.62	$3.95 \cdot 10^{-3}$	0.18
10	Brak Popr.	196	$1.27 \cdot 10^{-4}$	0.54	$2.76 \cdot 10^{-3}$	0.21
50	Znane Opt.*	10000	8066.76	60.56	$6.05 \cdot 10^{-3}$	0.51
50	Brak Popr.	384	8066.76	2.17	$5.64 \cdot 10^{-3}$	0.51
100	Znane Opt.*	10000	18703.25	49.16	$4.91 \cdot 10^{-3}$	1.13
100	Brak Popr.	833	18703.25	2.90	$3.49 \cdot 10^{-3}$	1.67

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.5.2 Zadanie 2: Funkcja Rosenbrock

Tabela 12: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,cuda}$	Najlepsze $f(x)$	T_{cuda} [s]	$T_{cuda}/i_{max,cuda}$ [s]	$S(n)$
10	Znane Opt.*	10000	$2.53 \cdot 10^{-6}$	61.17	$61.17 \cdot 10^{-3}s$	0.07
10	Brak Popr.	191	5.42	1.99	1.03 s	0.08
50	Znane Opt.*	10000	$9.44 \cdot 10^{-3}$	41.01	$41.01 \cdot 10^{-3}s$	0.52
50	Brak Popr.*	10000	$9.44 \cdot 10^{-3}$	38.15	$38.15 \cdot 10^{-3}$	0.57
100	Znane Opt.*	10000	10108.10	43.42	$43.42 \cdot 10^{-3}s$	1.05
100	Brak Popr.*	10000	10108.10	38.68	$38.68 \cdot 10^{-3}s$	1.09

*Algorytm osiągnął limit iteracji.

6.6 Wersja równoległa - MPI

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z 4-rdzeniowym procesorem. Tabela zawiera czasy wykonania dla wersji równoległej z wykorzystaniem MPI ($T_{mpi,r}$) wykorzystująca r rdzeni oraz obliczone przyspieszenie $S = T_{seq}/T_{mpi,r}$. Liczbę iteracji oznaczamy $i_{max,mpi}$.

6.6.1 Zadanie 1: Funkcja Schwefel

Tabela 13: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,mpi}$	Najlepsze $f(x)$	$T_{mpi,2}$ [s]	$T_{mpi,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.14	0.06	0.62	1.44
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	0.12	0.65	0.79	0.15
50	Znane Opt.*	10000	6726.75	19.17	20.45	1.60	1.50
50	Brak Popr.	357	6726.75	0.59	0.58	1.75	1.78
100	Znane Opt.*	10000	11854.13	38.22	36.52	1.45	1.52
100	Brak Popr.	1024	11854.13	3.83	3.87	1.56	1.54

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.6.2 Zadanie 2: Funkcja Rosenbrock

Tabela 14: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,mpi}$	Najlepsze $f(x)$	$T_{mpi,2}$ [s]	$T_{mpi,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.*	10000	5.42	3.58	5.59	1.15	0.74
10	Brak Popr.	191	5.42	0.09	0.11	0.89	0.73
50	Znane Opt.*	10000	115.20	13.10	13.88	1.64	1.55
50	Brak Popr.	9003	115.21	10.83	13.11	1.80	1.48
100	Znane Opt.*	10000	119.96	37.31	31.11	1.22	1.46
100	Brak Popr.*	10000	119.96	36.27	29.20	1.16	1.44

*Algorytm osiągnął limit iteracji.

7 Wnioski i analiza

7.1 Wnioski i analiza z etapu 1

7.1.1 Analiza wydajności i przyspieszenia - etap 1

W przeprowadzonych eksperymentach wersja równoległa algorytmu PSO okazała się **zawsze szybsza** od wersji sekwencyjnej (wartość $S > 1$ we wszystkich przypadkach).

- **Maksymalne przyspieszenie:** Najwyższą wartość $S \approx 2.53$ odnotowano dla funkcji Rosenbrocka przy $n = 10$ (Kryterium 1). Oznacza to ponad 2.5-krotne skrócenie czasu obliczeń.
- **Liczba rdzeni a przyspieszenie:** Mimo wykorzystania 6 procesów, przyspieszenie nie osiąga wartości bliskiej 6. Jest to spowodowane prawem Amdahla – zrównoleglona jest jedynie ewaluacja funkcji celu. Aktualizacja pozycji i prędkości oraz synchronizacja procesów (w tym kosztowna serializacja danych przy przesyłaniu dużych tablic między procesami) pozostają sekwencyjne lub generują narzut.
- **Wpływ wymiarowości:** Przy dużych wymiarach ($n = 100$) przyspieszenie jest zauważalne (ok. 1.2x - 1.7x), ale ograniczone przez zwiększony koszt przesyłania większych wektorów danych do procesów podrzędnych.

7.1.2 Zbieżność algorytmu w wysokich wymiarach - etap 1

Wyniki pokazują trudność optymalizacji w przestrzeniach wielowymiarowych ("przekleństwo wymiarowości"):

- Dla $n = 10$ algorytm radzi sobie dobrze, szczególnie dla funkcji Schwefela, gdzie znajduje optimum bliskie 0 ($1.27 \cdot 10^{-4}$).
- Dla $n = 50$ i $n = 100$ algorytm często utyka w minimach lokalnych. Dla funkcji Schwefela wartości $f(x)$ rzędu 6000-11000 świadczą o tym, że rój nie zdołał znaleźć globalnego minimum w tak dużej przestrzeni przy założonej liczbie iteracji. W takich przypadkach algorytm przerywał pracę po osiągnięciu limitu iteracji lub z powodu braku poprawy, co drastycznie wpływało na czas wykonania (np. 150 sekund dla Rosenbrocka sekwencyjnie przy $n = 100$).

Podsumowując, implementacja równoległa przynosi wymierne korzyści czasowe, szczególnie gdy algorytm musi wykonać dużą liczbę iteracji (np. przy trudnych problemach optymalizacyjnych, gdzie nie następuje szybka zbieżność).

7.2 Wnioski i analiza - etap 2

- **Wersja sekwencyjna** - Istotny wpływ na wyniki wersji sekwencyjnej miały również wprowadzone optymalizacje kodu.

Zrezygnowano z funkcji `apply_along_axis`, która mimo przynależności do biblioteki NumPy, w praktyce działała jak wolna pętla wykonywana w Pythonie. Zastąpienie jej operacjami bezpośrednio na macierzach pozwoliło wyeliminować narzut wywoływania funkcji dla każdej cząstki z osobna, co znacząco skróciło czas obliczania wartości funkcji celu.

Usprawniono mechanizm zatrzymywania algorytmu. Zastosowaliśmy kolejkę deque z ograniczonym rozmiarem, która działa jak „przesuwne okno”. Dzięki temu nie musimy przechowywać

i przetwarzać całej historii wyników od początku działania programu, a jedynie kilka ostatnich iteracji. To znacznie uprościło sprawdzanie, czy algorytm przestał znajdować lepsze rozwiązania.

Dodatkowo zysk wydajnościowy przyniosło przejście na operacje in-place w bibliotece NumPy. Zamiast tworzyć nowe kopie macierzy przy każdym obliczaniu prędkości czy nowej pozycji cząstki, modyfikujemy bezpośrednio te już istniejące w pamięci. Dzięki temu program nie traci czasu na ciągłe alokowanie nowej pamięci i kopiowanie dużych ilości danych, co przy tysiącach iteracji przełożyło się na wyraźne przyspieszenie obliczeń

- **Wersja równoległa - procesy** - wersja równoległa z wykorzystaniem procesów zaimplementowana w języku Python okazała się gorsza w każdym eksperymencie od wersji sekwencyjnej. Wynika to z dominowania operacji pamięciowych nad samymi obliczeniami. Wszystkie chunki są serializowane i kopiowane do oddzielnego procesu a wyniki łączone dopiero w procesie głównym. Również równolegle liczą się tylko wartości funkcji celu, a aktualizacje innych parametrów odbywają się już sekwencyjnie. W przyszłości można by rozważyć takie ulepszenia jak użycie asynchroniczności, aby zakończone procesy nie czekały na pozostałe lub wspólne fragmenty pamięci.
- **Wersja równoległa - CUDA** - Wersja GPU najlepiej skaluje się wraz ze wzrostem wymiarowości problemu. Dla $n = 100$ osiągnięto przyspieszenie przekraczające **1.5x** względem wersji sekwencyjnej. Dla małych problemów koszty transferu danych między CPU (host) a GPU (device) oraz inicjalizacji jąder CUDA przewyższają zysk z masowego przetwarzania równoległego, co czyni to podejście nieopłacalnym dla niskich wymiarów, co widać nie tylko w tym przypadku. W eksperymentach zaimplementowanie własnego jądra w bibliotece CuPy nie wpłynęło na przyspieszenie/spowolnienie obliczeń, dlatego operacje tego jądra mogły być zwykłym elementowym mnożeniem macierzy. Wykorzystanie wysokopoziomowej biblioteki ułatwiło pracę nad projektem i eksperymentami, ale na pewno wpłynęło na czas obliczeń, ze względu na brak niskopoziomowego dostępu do różnego rodzaju pamięci w karcie (globalna, współdzielona między wątkami w bloku, rejestrach itp.) oraz synchronizacji obliczeń między wątkami.
- **Wersja równoległa - wątki** - Wdrożenie wielowątkowości w wariancie no-GIL wykazało całkowitą nieopłacalność dla zadań o małej skali ($n = 10$). W tym zakresie czas operacji systemowych (tworzenie wątków, synchronizacja) jest nieproporcjonalnie długi względem błyskawicznych operacji sekwencyjnych. W efekcie wersja ta zamiast przyspieszać, wprowadza gigantyczny narzut, czyniąc ją znacznie wolniejszą od prostego kodu sekwencyjnego.

Sytuacja zmienia się diametralnie po przekroczeniu progu złożoności przy wymiarze $n = 100$. Dopiero tutaj algorytm osiąga punkt opłacalności, notując przyspieszenie powyżej jedności ($S > 1.0$), co widać szczególnie przy funkcji Schwefela ($S \approx 1.47$). Dowodzi to, że zniesienie blokady GIL pozwala na efektywne wykorzystanie wielordzeniowości, o ile czas trwania pojedynczej iteracji jest wystarczająco długi, by zamortyzować koszty systemowe.

Widać to również w skalowalności rozwiązania. Przy małych wymiarach zwiększenie liczby wątków z dwóch do czterech często pogarsza wyniki przez wzrost rywalizacji o zasoby. Natomiast przy $n = 100$ tendencja się odwraca i dodatkowe wątki przynoszą wymierny zysk. Oznacza to, że implementacja wątkowa z pamięcią współdzieloną jest rekomendowana wyłącznie dla zadań o dużej gęstości obliczeniowej.

- **Wersja równoległa - MPI** - Większość eksperymentów dla $n = 50$ i $n = 100$ wykazała wyraźne przyspieszenie ($S > 1.4$, dochodzące do 1.8), co jest znaczącą poprawą względem wersji procesowej, która w tych samych warunkach notowała spowolnienie ($S < 0.8$). Kluczowa była zmiana zarządzania pamięcią na rozproszony. W przeciwieństwie do wersji z pulą procesów, gdzie w każdej iteracji przesyłane są pełne tablice pozycji, w MPI procesy przechowują dane lokalnie i wymieniają się jedynie kilkoma liczbami (wartościami najlepszego rozwiązania). Dzięki temu narzut komunikacyjny stał się pomijalny w stosunku do zysku wynikającego z równoległego przetwarzania.

Warto jednak zauważyć granicę opłacalności rozwiązania. Dla najniższych wymiarów ($n = 10$), czas trwania pojedynczej iteracji jest na tyle krótki, że narzut związany z synchronizacją barierową (**Barrier**) (punkt w kodzie, w którym wstrzymywane jest wykonywanie programu, aż dotrą do niego wszystkie procesy) i komunikacją międzyprocesową przeważa nad czasem obliczeń. Skutkuje to brakiem przyspieszenia lub nawet spowolnieniem ($S < 1$ dla $n=10$ w kilku przypadkach). Pełny potencjał MPI jest widoczny przy bardziej złożonych obliczeniowo zadaniach ($n \geq 50$).

- **Wektoryzacja wygrywa z równoległością:** Największy skok wydajnościowy uzyskano nie poprzez operacje równoległe, ale poprzez optymalizację sekwencyjną - usunięcie pętli w Pythonie na rzecz operacji wektorowych NumPy (SIMD) oraz operacji *in-place*. Dla małych wymiarów ($n < 50$) jest to naprawdę słuszne podejście.

Projekt wykazał, że równoległe nie zawsze oznacza szybciej. Kluczem do wydajności jest dopasowanie modelu równoległości do charakterystyki i rozmiaru przetwarzanych danych.

Kolejnym ważnym aspektem podczas korzystania ze zrównoleglenia/optymalizacji jest ciągle wykonywanie eksperymentów. Dywagacje teoretyczne czy nieprzemyślane zrównoleglenie (podejście im więcej wątków tym lepiej) zwykle nie przynoszą pozytywnych skutków, kiedy w grę wchodzi optymalizacje przygotowane przez autorów bibliotek, interpreterów, kompilatorów czy producentów sprzętu na którym wykonuje się obliczenia, inne warunki mogą przynieść inne wyniki.

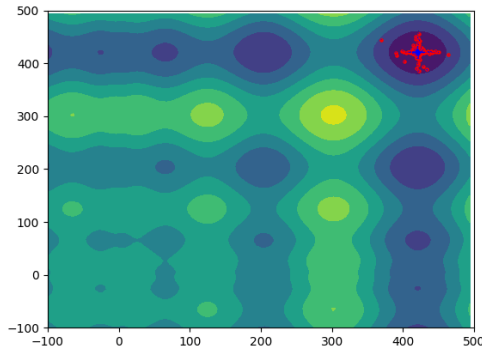
8 Wizualizacja zbieżności algorytmu ($n=2$)

W celu zobrazowania działania algorytmu PSO oraz dynamiki poruszania się roju, przeprowadzono symulację dla przestrzeni dwuwymiarowej ($n = 2$). Pozwoliło to na graficzne przedstawienie pozycji cząstek na płaszczyźnie wyznaczonej przez granice funkcji celu.

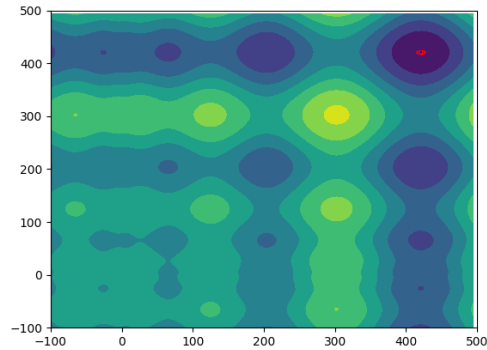
Poniższe rysunki przedstawiają **ostatnią iterację** algorytmu, czyli stan końcowy, w którym proces został zatrzymany przez jedno z kryteriów stopu. Pełne animacje, ukazujące proces zbieżności klatka po klatce, zostały dołączone do sprawozdania jako osobne pliki w formacie GIF.

8.1 Funkcja Schwefela

Na Rysunku 1 przedstawiono końcowe położenie cząstek dla funkcji Schwefela.



(a) Kryterium 1: Znane optimum



(b) Kryterium 2: Brak poprawy

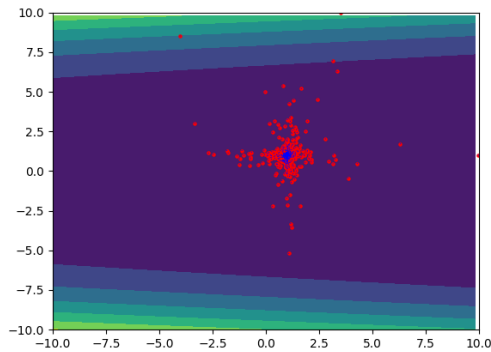
Rysunek 1: Końcowy stan roju dla funkcji Schwefela ($n = 2$).

Analiza wizualna:

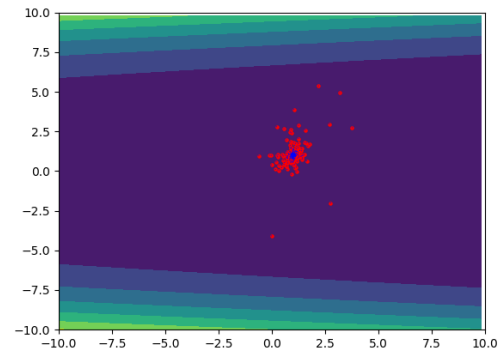
- **Obraz po lewej (Znane optimum):** Algorytm zbiegł do globalnego minimum, jednak widoczny jest pewien rozrzut cząstek wokół punktu optymalnego. Wynika to z natury kryterium stopu ($\|g_{best} - x_{opt}\| \leq 10^{-3}$) – symulacja jest przerywana natychmiast, gdy najlepsza cząstka (g_{best}) osiągnie zadaną bliskość celu. Pozostałe cząstki, które podążały za liderem, mogły nie zdążyć w pełni skupić się w tym samym punkcie przed zatrzymaniem algorytmu.
- **Obraz po prawej (Brak poprawy):** Widoczne jest **znacznie silniejsze skupienie** cząstek w punkcie minimum. Jest to efekt działania drugiego kryterium: algorytm musiał wykonać 50 dodatkowych iteracji, w których zmiana funkcji celu była znikoma ($\leq 10^{-6}$). Ten "czas stabilizacji" pozwolił całej populacji na dociągnięcie do pozycji lidera i utworzenie bardzo zwartej grupy.

8.2 Funkcja Rosenbrocka

Na Rysunku 2 przedstawiono wyniki dla funkcji Rosenbrocka.



(a) Kryterium 1: Znane optimum



(b) Kryterium 2: Brak poprawy

Rysunek 2: Końcowy stan roju dla funkcji Rosenbrocka ($n = 2$).

Analiza wizualna:

- **Obraz po lewej (Znane optimum):** Rój skutecznie zlokalizował obszar minimum w "dolinie Rosenbrocka" (punkt 1, 1). Podobnie jak w poprzednim zadaniu, natychmiastowe przerwanie pętli po osiągnięciu warunku odległości sprawia, że część cząstek jest jeszcze w fazie eksploracji lokalnego otoczenia optimum, tworząc luźniejszą chmurę.
- **Obraz po prawej (Brak poprawy):** Sytuacja wygląda analogicznie jak w poprzednim zadaniu. Cząstki tworzą tutaj znacznie gęstsze skupisko niż w przypadku pierwszego kryterium. Jest to efekt dodatkowych 50 iteracji, które algorytm musiał wykonać, by potwierdzić brak poprawy wyniku – ten czas pozwolił reszcie roju na dociągnięcie do lidera i zebranie się w jednym punkcie.